Reliable Data Acquisition Error Handling - Integration Error in Salesforce Object

Overview

Business Context

The enterprise currently processes critical business transactions (e.g., Patient Data, Orders) through the MuleSoft platform. When errors occur, the current process requires significant manual intervention from the IT Platform Team to investigate logs, extract payloads, and explain failures to business stakeholders. This creates a bottleneck and delays data remediation.

Objective

The objective of this design is to implement a **Robust, Automated, and Self-Service Error Handling Framework** within the Mulesoft and Salesforce. This framework achieves two primary goals:

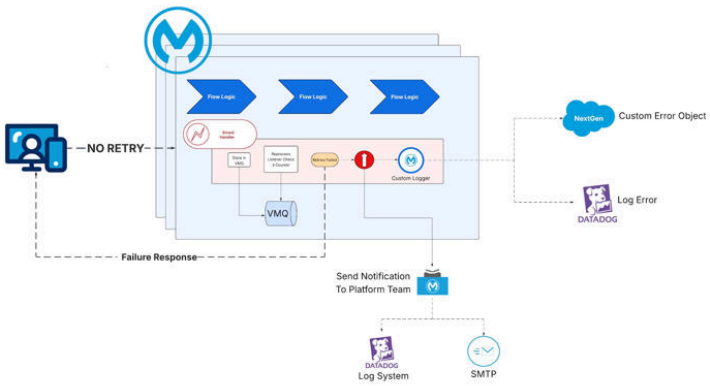
1. **Reliability:** Guarantees zero message loss for system failures (network/infrastructure) via automated retries and Dead Letter Queues (DLQ).
2. **Self-Service:** Offloads "Bad Data" issues directly to business owners by logging readable error contexts and payloads into Salesforce Custom Error Object (`Integration_Error__c`), enabling them to view and fix data without IT involvement., kind of self-serve model of the troubleshooting the integration failures

Scope

- **In Scope:** All Process Layer APIs handling asynchronous message ingestion (Anypoint MQ).
- **In Scope:** Salesforce Custom Object Design and MuleSoft Salesforce Connector implementation.
- **Out of Scope:** Experience Layer synchronous error handling (which returns immediate HTTP 400s to callers).

High-Level Architecture

The solution adheres to **API-Led Connectivity** principles, specifically focusing on the **Asynchronous Process Layer**. The architecture decouples message acceptance from message processing to ensure high throughput and reliability.



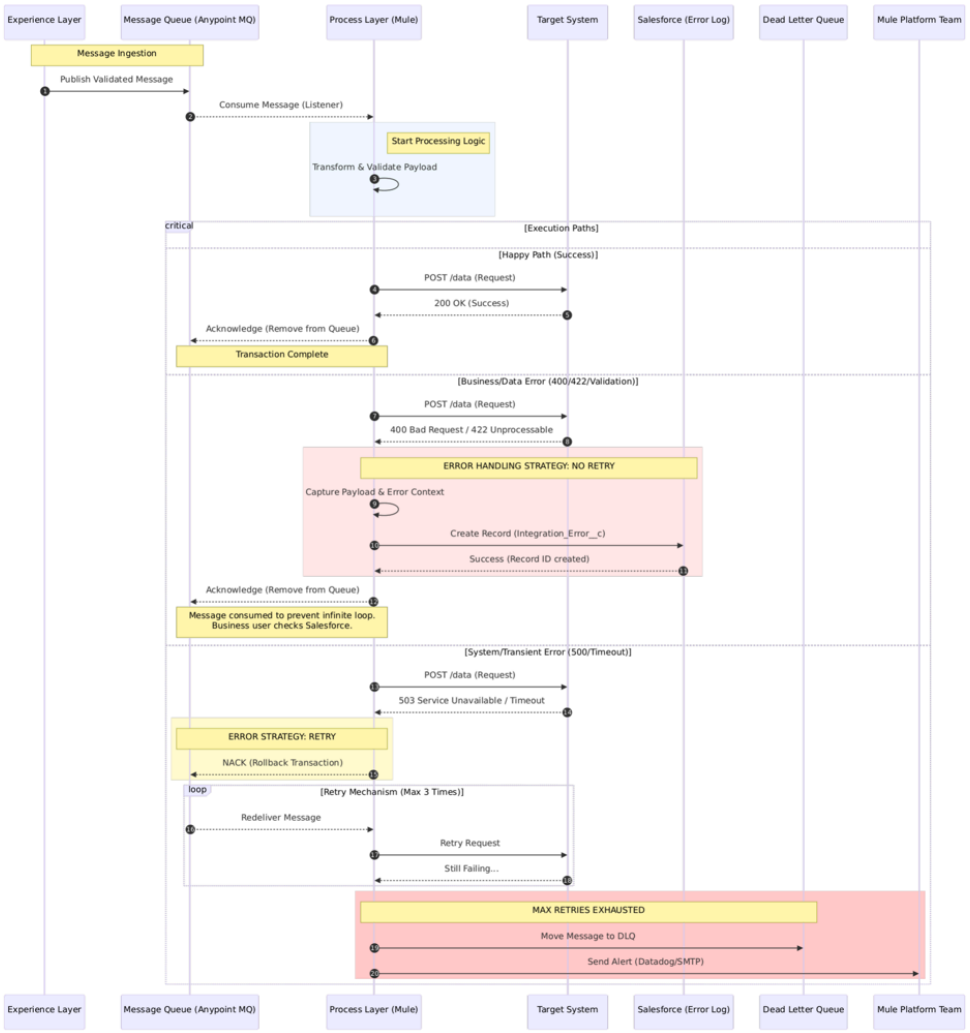
Architectural Pattern: "Reliable Acquisition"

The pattern follows a "Store → Process" methodology:

1. **Ingest:** The API receives a message and immediately persists it to a durable queue (Anypoint MQ).
2. **Process:** A separate flow listener picks up the message for processing.
3. **Differentiate:** The logic distinguishes strictly between "Business/Data Errors" and "System/Transient Errors."

Sequence Diagram

The following interaction diagram details the exact message lifecycle across the Experience Layer, Process Layer, Target Systems, and Salesforce.



Comprehensive Error Handling Strategy

This section details the logic governing how the MuleSoft Global Error Handler routes exceptions.

The "No-Reprocess" Philosophy for Data Errors

Principle: If a message fails due to invalid data (e.g., "Invalid Patient Data"), retrying it is futile and wasteful. It will fail 100% of the time.

Actions:

1. **Intercept** the error.
2. **Log** the *entire* payload and error message to the system of engagement (Salesforce).
3. **Consume (ACK)** the message from the queue. This prevents the queue from getting blocked by "poison messages."

The "Reliability" Philosophy for System Errors

Principle: If a message fails due to infrastructure (e.g., "Database Locked"), it is likely temporary.

Action:

1. **Rollback (NACK)** the transaction. This forces the message to stay in the queue.
2. **Retry** automatically based on the connector's Redelivery Policy.
3. **Escalate** to a Dead Letter Queue (DLQ) only after retries are exhausted.

Error Code Mapping Table

Mule Error Type	Description	HTTP Status	Classification	Action
`MULE:EXPRESSION`	DataWeave transformation failure	500	**Business**	Log to SFDC & ACK
`VALIDATION:INVALID_BOOLEAN`	Schema validation failure	400	**Business**	Log to SFDC & ACK
`HTTP:BAD_REQUEST`	Target system rejected data format	400	**Business**	Log to SFDC & ACK
`HTTP:UNAUTHORIZED`	Invalid credentials (config issue)	401	**System**	Retry & DLQ
`HTTP:FORBIDDEN`	Access denied	403	**System**	Retry & DLQ
`HTTP:NOT_FOUND`	Resource/ID does not exist	404	**Business**	Log to SFDC & ACK
`HTTP:TIMEOUT`	Target system slow/unresponsive	504	**System**	Retry & DLQ
`HTTP:CONNECTIVITY`	Network down	-	**System**	Retry & DLQ
`HTTP:SERVICE_UNAVAILABLE`	Target system maintenance	503	**System**	Retry & DLQ
`DB:CONNECTIVITY`	Database down	-	**System**	Retry & DLQ
`DB:QUERY_EXECUTION`	Bad SQL syntax or Constraint	-	**Business**	Log to SFDC & ACK

Salesforce Design: The "Self-Service" Interface

The `Integration_Error__c` object is the heart of the business visibility strategy. It transforms cryptic log lines into actionable business records.

Object Schema Specification

Object Label: Integration Error Log

API Name: `Integration_Error__c`

Field Label	API Name	Data Type	Length	Required	Detailed Description
Error ID	`Error_ID__c`	Auto Number	-	Yes	System generated ID (e.g., ERR-1001) for ticket reference.
Mule App Name	`Mule_App_Name__c`	Text	100	Yes	Name of the MuleSoft application (e.g., `proc-patient-sync-api`).
Correlation ID	`Correlation_ID__c`	Text	255	Yes	The global transaction ID for cross-referencing with Splunk/Datadog logs.
Timestamp	`Error_Timestamp__c`	Date/Time	-	Yes	Exact UTC timestamp of the failure.
Error Message	`Error_Message__c`	Long Text	32,768	Yes	A human-readable summary of the error (e.g., "Field 'Phone' is required").
Payload Data	`Payload__c`	Long Text	131,072	Yes	**CRITICAL FIELD.**Contains the full JSON/XML request body. Increased size to handle large payloads.
Response Data	`Response_Data__c`	Long Text	32,768	No	The specific error response returned by the downstream system (e.g., Oracle error codes).
Resolved Flag	`Resolved__c`	Picklist	-	Yes	Values: `New`, `In Review`, `Fixed`, `Ignored`. Defaults to `New`.
Resolved By	Resolved_By__c	Lookup(User)	-	Yes	User who fixed the issue

Resolution Notes	`Resolution_Notes__c`	Long Text	32,768	No	Notes entered by the business user describing how they fixed the data.
Error Type	`Error_Type__c`	Text	100	Yes	SYSTEM, DATA, BUSINESS etc.. Allows filtering errors by type.
API Version	API_Version__C	Text	10	Yes	v2.1 etc. to distinguish from multiple versions of the same API
External Ref ID	`External_Ref_ID__c`	Text	255	No	The primary business key (e.g., PatientID, OrderNumber) for easier searching.
Record Creation Date	CreatedDate	System	-	-	Standard SF System Field

Security & Access Control for this Object

- MuleSoft Integration User in Salesforce with Appropriate Profile, Object/Field Create/Update/Delete permissions
- **Read/Edit from Salesforce UI:** Business Data Stewards, Application Support Team.

Technical Implementation Specifications

This section provides the exact code samples/artifacts required for developers to implement this standard.

Queue Listener Configuration

To support the System Error Retry logic, the listener must be configured with a **Redelivery Policy**.

XML

> `<vm:listener config-ref="VM_Config" queueName="InputQueue" numberOfConsumers="1">`
> `    <vm:redelivery-policy maxRedeliveryCount="3" useSecureHash="true"/>`
> `</vm:listener>`

Global Error Handler (XML)

This is the standard `error-handler` block to be included in the common library or template.

XML

```
<error-handler name="Global_Error_Handler">
    <on-error-continue type="VALIDATION:ALL, DATAWEAVE:ERROR, HTTP:BAD_REQUEST, HTTP:NOT_FOUND, HTTP:PARSING, DB:QUERY_EXECUTION" enableNotifications="true" logException="true">
        <logger level="ERROR" message="Business Error Detected. Initiating Salesforce Logging. CorrelationID: #[correlationId]"/>
        <set-variable value="#[error.description]" variableName="errorDescription"/>
        <set-variable value="#[if (error.errorMessage.payload != null) error.errorMessage.payload else 'No Response']" variableName="errorResponse"/>
        <ee:transform doc:name="Map to SF Object">
            <ee:message>
                <ee:set-payload><![CDATA[
                    %dw 2.0
                    output application/json
                    ---
                    [{
                        "Mule_App_Name__c": app.name,
                        "Correlation_ID__c": correlationId,
                        "Error_Timestamp__c": now(),
                        "Mule_Environment__c": p('mule.env'),
                        "Error_Message__c": vars.errorDescription,
                        // Convert payload to string to fit in Text Area
                        "Payload__c": write(vars.originalPayload, "application/json"),
                        "Response_Data__c": write(vars.errorResponse, "application/json"),
                        "External_Ref_ID__c": vars.businessKey default "Unknown",
                        "Status__c": "New"
```

```
                    }]
                ]]></ee:set-payload>
            </ee:message>
        </ee:transform>
        <salesforce:create type="Integration_Error__c" config-ref="Salesforce_Config" doc:name="Create Error Record"/>
        <logger level="INFO" message="Successfully logged error to Salesforce. Message will be acknowledged."/>
        </on-error-continue>


    <on-error-propagate type="CONNECTIVITY, HTTP:TIMEOUT, HTTP:SERVICE_UNAVAILABLE, DB:CONNECTIVITY, LOCK:TIMEOUT">

        <logger level="WARN" message="Transient System Error. Rolling back transaction for retry. CorrelationID: #[correlationId]"/>
        </on-error-propagate>
    <on-error-propagate type="ANY">
        <logger level="ERROR" message="Unknown Error occurred. Defaulting to Retry strategy. Error: #[error.description]"/>
    </on-error-propagate>
</error-handler>
```

Dead Letter Queue (DLQ) Handling

When the `maxRedeliveryCount` (3) is exceeded, the MQ connector will automatically reject the message. To capture this:

1. **Configuration:** Configure the Input Queue to have a specific DLQ assigned (e.g., `Patient.Queue.DLQ`).
2. **Notification:** A separate Mule flow must listen to `Patient.Queue.DLQ`.
   - **Action:** When a message arrives here, it means it failed 3 retries.
   - **Alert:** Send an email/Slack notification to the Platform Admin team immediately. "Urgent: Message moved to DLQ."

Operational Procedures & Workflows

Workflow A: Business User (Data Remediation)

- **Trigger:** User receives a daily digest email from Salesforce listing new `Integration_Error__c` records.
- **Action:**
  1. Log in to Salesforce.
  2. Open the `Integration Errors` tab.
  3. Review the `Error Message` (e.g., "PatientID 555 not found").
  4. Review the `Payload` field to see the data sent.
  5. **Fix:** Go to the Source system and create PatientID 555.
  6. **Close:** Change the Salesforce record status to `Resolved`.
- **Outcome:** The data issue is resolved without IT knowing the error ever happened. The source system will likely re-trigger the update in the next batch, or the user manually re-triggers.

Workflow B: Platform Support (System Remediation)

- **Trigger:** Datadog/SMTP Alert P1: "DLQ Depth > 0".
- **Action:**
  a. Check Anypoint Monitoring.
  b. Identify the root cause (e.g., "Okta was down for patching").
  c. Verify the Okta is back up.
  d. **Recover:** Execute the **DLQ Replay Script** (a Mule utility API) that reads messages from the DLQ and places them back into the Main Input Queue.
- **Outcome:** Messages are processed successfully. No data is lost.

Monitoring & Observability

Datadog/Mulesoft Logs and Dashboards

- All logs must include `correlationId`.
- **Metric:** `mule.message.error_count` tagged by `error_category` (Business/System).
- **Alerts:**
  - *High Urgency:* Any message entering a DLQ.
  - *Low Urgency:* High volume of Business Errors (indicates an issue).

Salesforce Dashboard

A custom dashboard "MuleSoft Integration Health" can be created in Salesforce containing:

- **Chart:** Errors by Application (Last 7 Days).
- **List:** Unresolved Errors (Status = New).
- **Metric:** Average Resolution Time.
